La examen sunt permise orice materiale tiparite sau scrise de mana.

(1) Clasele de complexitate ale algoritmilor:

-> de stiut ce sunt, pentru o functie f:N->[0,infinit), ce sunt: clasa functiilor asimptotic mai mici sau egale cu f, clasa functiilor asimptotic mai mare sau egale cu f si clasa functiilor asimptotic egale cu f;

-> de calculat clasa de complexitate a unui algoritm, i.e. clasa de complexitate a functiei T:N-> N care da timpul de executie al algoritmului in functie de dimensiunea n a input-ului;

-> de stiut, orientativ, ce spune Teorema Master.

(2) Metode de sortare; de stiut ce inseamna o sortare stabila:

2.1. Sortarea prin interschimbare (sortarea prin metoda bulelor: bubblesort); la fel ca la sortarea prin selectie, in cazul unei sortari crescatoare, daca parcurgerile vectorului se fac de la dreapta la stanga, atunci se vor impinge, pe rand, elementele cele mai mari la dreapta vectorului, iar, daca parcurgerile vectorului se fac de la stanga la dreapta, atunci se vor impinge, pe rand, elementele cele mai mici la stanga vectorului; asadar avem si varianta combinata cu sortarea prin selectie.

2.2. Sortarea prin insertie (insertsort); variante: prin insertie in stanga/dreapta vectorului.

2.3. Sortarea prin selectie (selectsort); variante: prin selectia minimului/maximului.

2.4. Sortarea prin numarare (countingsort); varianta generala, nu doar cazul particular al valorilor doua cate doua distincte.

2.5. Sortarea prin interclasare (binara) (mergesort): varianta recursiva, iar, dupa studiul stivelor, si cea iterativa.

2.6. Sortarea rapida (quicksort), cu doua dintre metodele posibile de partitionare, cu alegerea unui pivot: varianta recursiva, iar, dupa studiul stivelor, si cea iterativa.

2.7. Sortarea cu ansamble (heapsort), dupa studiul ansamblelor.

2.8. Sortarea cu arbori binari de cautare (searchtreesort), dupa studiul acestora.

(3) Structuri de date liniare: liste, in reprezentare statica (sub forma de vectori) si inlantuita; ne intereseaza in special:

3.1. Stive (liste de tipul LIFO: last in first out; exemplu clasic: stiva de farfurii).

3.2. Cozi (liste de tipul FIFO: first in first out; exemplu clasic: coada de la magazin).

La structurile de date neliniare intra si, de exemplu, matricile (i.e. vectorii bidimensionali) si, in general, vectorii multidimensionali.

(4) Structuri de date neliniare: grafuri (1-multigrafuri, daca folosim terminologia generala, i.e. grafuri fara muchii/arce multiple, adica grafuri in care intre oricare doua noduri/varfuri exista cel mult o muchie/un arc) neorientate/orientate. Grafuri ponderate; amintesc ca orice graf poate fi privit ca graf ponderat, cu ponderea canonica, i.e. ponderea fiecarei muchii/fiecarui arc egala cu 1. Reprezentari pentru grafuri neorientate/orientate neponderate/ponderate:

-> matricea de adiacenta/ponderilor muchiilor/arcelor;

-> listele (statice/inlantuite) de vecini/succesori, cu ponderi pentru muchiile/arcele corespunzatoare in cazul ponderat;

-> lista (statica/inlantuita) muchiilor/arcelor, cu ponderi in cazul ponderat.

4.1. Parcurgerile grafurilor:

-> in adancime (depth first: dupa vizitarea unui nod, sunt parcursi imediat vecinii sai nevizitati), recursiv si iterativ, folosind o stiva;

-> in latime (breadth first: in ordinea crescatoare a distantelor fata de nodul/varful de start, distante date de numarul de muchii/arce al unui lant/drum minimal de la nodul/varful de start), iterativ, folosind o coada.

4.2. Determinarea componentelor conexe si, in cazul grafurilor orientate, a componentelor tare conexe.

4.3. Determinarea distantelor si lanturilor/drumurilor minime in grafuri neponderate/ponderate:

-> dintr-un nod: prin parcurgerea in latime incepand din acel nod, iar, dupa studierea arborilor, cu algoritmul lui Dijkstra pentru determinarea unui arbore al lanturilor minime din acel nod in cazul neorientat (arbore partial al componentei conexe a nodului respectiv in cazul neorientat);

-> intre oricare doua noduri: cu algoritmul Roy-Floyd-Warshall.

4.4. Determinarea unui arbore economic (i.e. arbore partial de cost minim) al unui graf neorientat conex ponderat; pentru un graf neorientat ponderat neconex, putem determina o

padure de arbori economici ai componentelor sale conexe:

-> algoritmul lui Prim;

-> algoritmul lui Kruskal, inclusiv varianta optimizata cu min-ansamblul muchiilor.

(5) Caz particular de grafuri: arbori (grafuri neorientate conexe aciclice; a se vedea si caracterizarile acestora). Organizarea arborilor pe niveluri, cu un nod radacina. Caz particular de arbori: arborii binari. Reprezentarile statice/inlantuite:

-> pentru arborii oarecare: cu vectorul tata/legatura catre nodul tata, cu vectorii/legaturile fiu, frate, eventual cu nodul radacina retinut separat;

-> pentru arborii binari: cu vectorii/legaturile fiu stang, fiu drept, eventual cu nodul radacina retinut separat.

Ca instrumente utile:

-> afisarea "grafica" a unui arbore binar crescand din stanga ecranului, pentru a-i vedea structura;

-> construirea unei expresii asociate unui arbore oarecare/binar, pentru a-i retine structura si, eventual, a face mai usor, "pe hartie", parcurgerile acestor arbori.

5.1. Parcurgerile unui arbore oarecare, efectuate recursiv si iterativ, ca la grafuri, dar fara a necesita marcarea nodurilor vizitate, intrucat nu avem cicluri:

-> in adancime incepand din radacina;

-> in latime incepand din radacina, adica pe niveluri.

Parcurgerile unui arbore binar, efectuate recursiv si iterativ:

-> in preordine;

-> in inordine;

-> in postordine.

5.2. Arbori binari de cautare (nestricti la stanga/dreapta) si sortarea cu arbori binari de cautare (searchtreesort-ul): inserarea elementelor de sortat intr-un astfel de arbore urmata de parcurgerea arborelui in inordine.

(6) Caz particular de arbori: ansamble (arbori binari completi pana la ultimul nivel, pe care nodurile sunt grupate la stanga, astfel ca un ansamblu poate fi reprezentat printr-un vector

a[1..n] in care, pentru orice 1<=i<=n:

-> tatal lui a[i] este a[i/2], daca i>1, altfel a[i] = a[1] = radacina ansamblului, care nu are nod tata,

-> fiul stang al lui a[i] este a[2*i], daca 2*i<=n, iar fiul drept al lui a[i] este a[2*i+1], daca 2*i+1<=n, altfel nu exista fiul respectiv,

si in care:

-> in cazul unui min-ansamblu, informatia din fiecare nod este mai mica sau egala decat cea din fiecare fiu al sau,

-> in cazul unui max-ansamblu, informatia din fiecare nod este mai mare sau egala decat cea din fiecare fiu al sau.

Inserarea intr-un ansamblu: pe ultima pozitie dupa incrementarea dimensiunii ansamblului cu o unitate, urmata de promovarea nodului de pe acea noua pozitie in ansamblu.

Extragerea dintr-un ansamblu (decapitarea ansamblului): din radacina, urmata de mutarea ultimului element in locul radacinii, decrementarea dimensiunii ansamblului cu o unitate, apoi retrogradarea noii radacini.

6.1. Sortarea cu ansamble (heapsort-ul): inserarea succesiva a valorilor de sortat intr-un ansamblu, urmata de decapitarile succesive ale acestuia pana la extragerea tuturor valorilor:

-> cu min-ansamble => sortare crescatoare;

-> cu max-ansamble => sortare descrescatoare.

Facultativ, a se vedea si solutia propusa de un student: sortare crescatoare folosind un max-ansamblu, in care valorile maxime extrase prin decapitari succesive sunt depuse la sfarsitul ansamblului, astfel ca vectorul sortat crescator este retinut in vectorul in care fusese construit ansamblul.

6.2. Algoritmul lui Huffman: metoda generala pentru minimizarea lungimii externe ponderate a unui arbore cu ponderi atasate nodurilor frunza; aplicatii:

-> interclasarea optimala a mai multor vectori ordonati;

-> codificare Huffman: pentru un vector de caractere cu frecvente diferite de aparitie in mesaje/fraze, determinarea unui cod binar cu proprietatea prefixului astfel incat lungimile mesajelor codificate sa fie cat mai mici.